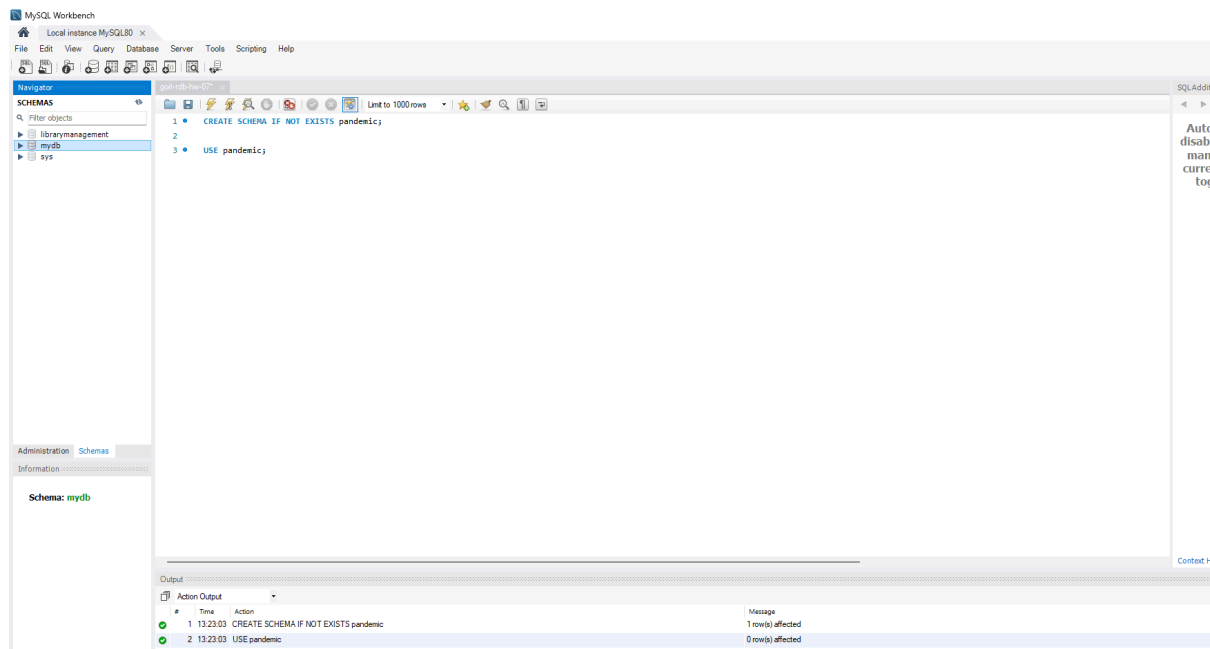
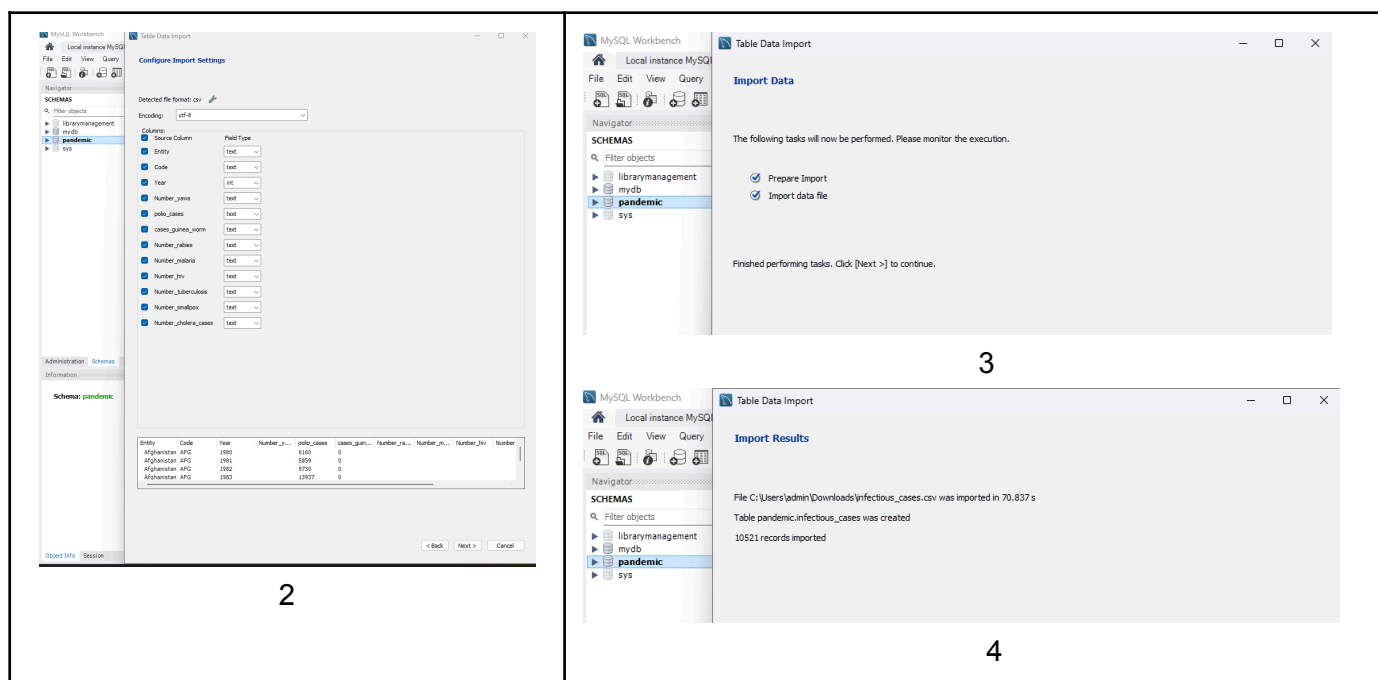


Фінальний проект до курсу “Реляційні бази даних”.

1. Завантажте дані:



ріс.1 Створено схему pandemic у базі даних та обрано її як схему за замовчуванням за допомогою SQL-команди.



ріс.2,3,4. Імпортування даних за допомогою Import wizard.

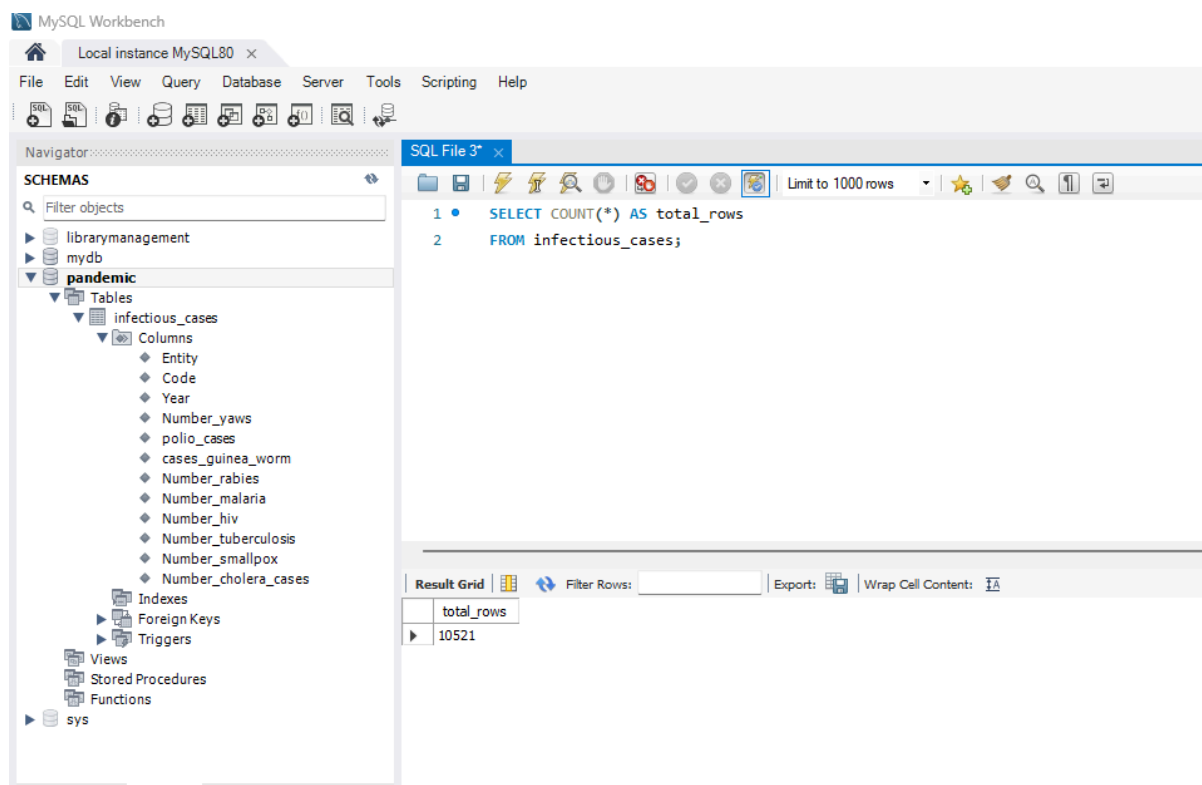


рис.5 Перевірка кількості рядків. Базовий контроль якості даних (Data Quality Check)

- дані завантажені повністю
- втрат при імпорті немає
- можна безпечно будувати логіку поверх даних

Крок 2. Нормалізуйте таблицю `infectious_cases` до 3ї нормальної форми. Збережіть у цій же схемі дві таблиці з нормалізованими даними.

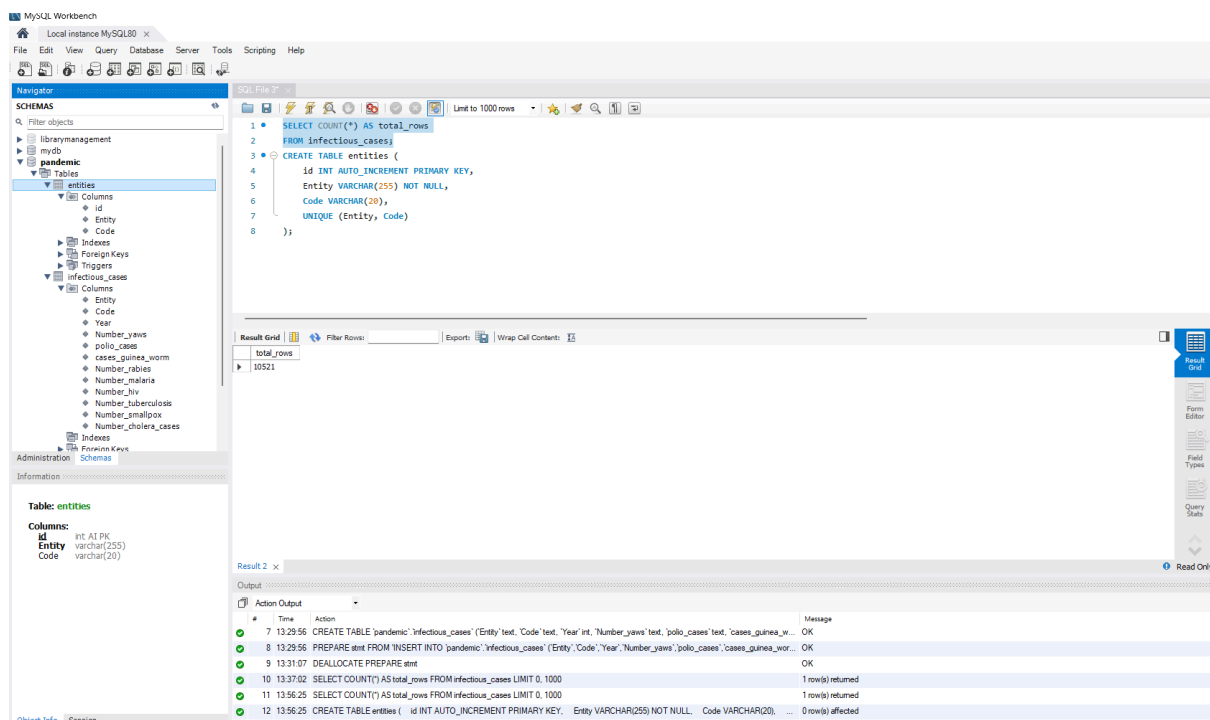
Дивимось дані, щоб бути у контексті:

- атрибути `Entity` та `Code` постійно повторюються. Позбудемось цього за допомогою нормалізації даних.

3NF (Third Normal Form — третя нормальна форма) означає:

- прибираємо дублювання;
- розділяємо сутності.

Розділяємо на 2 таблиці:

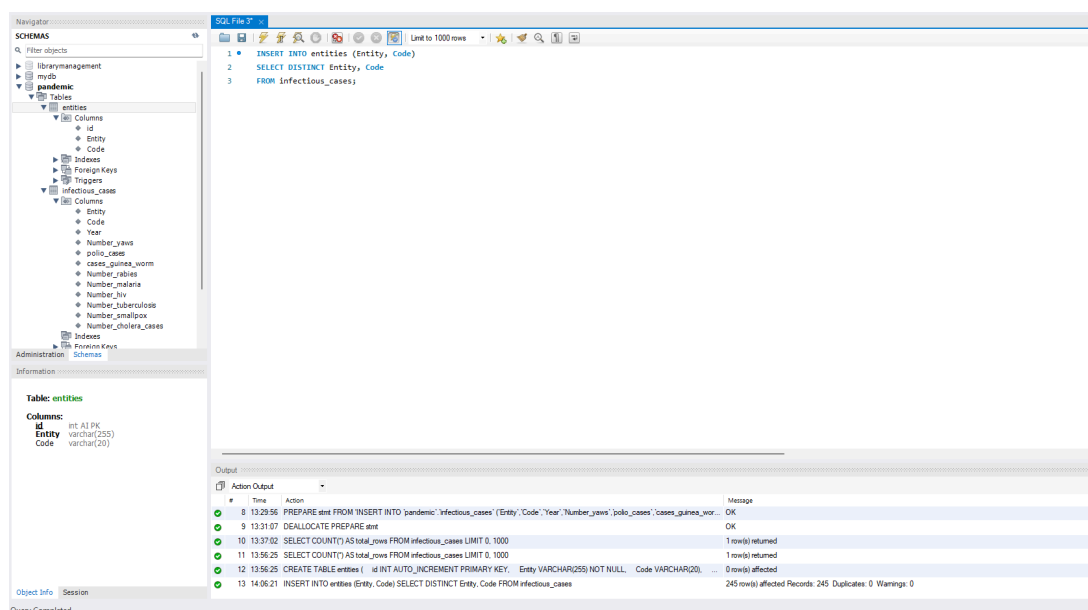


ріс.6 **entities** (довідник країн) id | Entity | Code

Робимо це для того щоб:

- винести повторювані дані
- створити основу для зв'язку (Foreign Key)
- підготувати структуру під нормалізацію

Заповнення таблиці entities. Зараз таблиця порожня. Ми маємо взяти: Entity + Code з основної таблиці, але без дублікатів, для цього використовується: SELECT DISTINCT



ріс.7 З 10521 рядка залишиться ~200–300 унікальних записів, кожна країна/регіон буде лише один раз

The screenshot shows the MySQL Workbench interface. The left sidebar displays the database schema, including tables like 'entities' and 'infectious_cases'. The main window shows a query result for 'entities_count' with 245 rows. The bottom panel shows the 'Action Output' log, indicating that 245 rows were affected by the query.

Table: entities

Columns:

- id: int AI PK
- Entity: varchar(255)
- Code: varchar(20)

Result 3:

#	Time	Action	Message
9	13:31:07	DEALLOCATE PREPARE stmt	OK
10	13:37:02	SELECT COUNT(*) AS total_rows FROM infectious_cases LIMIT 0, 1000	1 row(s) returned
11	13:56:25	SELECT COUNT(*) AS total_rows FROM infectious_cases LIMIT 0, 1000	1 row(s) returned
12	13:56:25	CREATE TABLE entities (id INT AUTO_INCREMENT PRIMARY KEY, Entity VARCHAR(255) NOT NULL, Code VARCHAR(20), ...	0 row(s) affected
13	14:06:21	INSERT INTO entities (Entity, Code) SELECT DISTINCT Entity, Code FROM infectious_cases	245 row(s) affected Records: 245 Duplicates: 0 Warnings: 0
14	14:07:46	SELECT COUNT(*) AS entities_count FROM entities LIMIT 0, 1000	1 row(s) returned

рис.8 Перевірка. 245 унікальних записів → нормалізація йде правильно.

- дублікати прибрані
- довідник сформований
- структура відповідає **3NF (Third Normal Form — третя нормальна форма)**

The screenshot shows the MySQL Workbench interface. The left sidebar displays the database schema, including the newly created table 'infectious_cases_normalized'. The main window shows the SQL query used to create this table. The bottom panel shows the 'Action Output' log, indicating that the table was created successfully.

Table: infectious_cases_normalized

Columns:

- id: int AI PK
- entity_id: int
- Year: text
- Number_yaws: text
- polio_cases: text
- cases_guinea_worm: text
- Number_rabies: text
- Number_malaria: text
- Number_hiv: text
- Number_tuberculosis: text
- Number_smallpox: text
- Number_cholera_cases: text

SQL Query:

```

1 CREATE TABLE infectious_cases_normalized (
2   id INT AUTO_INCREMENT PRIMARY KEY,
3   entity_id INT NOT NULL,
4   Year INT NOT NULL,
5   Number_yaws TEXT,
6   polio_cases TEXT,
7   cases_guinea_worm TEXT,
8   Number_rabies TEXT,
9   Number_malaria TEXT,
10  Number_hiv TEXT,
11  Number_tuberculosis TEXT,
12  Number_smallpox TEXT,
13  Number_cholera_cases TEXT,
14  FOREIGN KEY (entity_id) REFERENCES entities(id)
15 );

```

Action Output:

#	Time	Action	Message
10	13:37:02	SELECT COUNT(*) AS total_rows FROM infectious_cases LIMIT 0, 1000	1 row(s) returned
11	13:56:25	SELECT COUNT(*) AS total_rows FROM infectious_cases LIMIT 0, 1000	1 row(s) returned
12	13:56:25	CREATE TABLE entities (id INT AUTO_INCREMENT PRIMARY KEY, Entity VARCHAR(255) NOT NULL, Code VARCHAR(20), ...	0 row(s) affected
13	14:06:21	INSERT INTO entities (Entity, Code) SELECT DISTINCT Entity, Code FROM infectious_cases	245 row(s) affected Records: 245 Duplicates: 0 Warnings: 0
14	14:07:46	SELECT COUNT(*) AS entities_count FROM entities LIMIT 0, 1000	1 row(s) returned
15	14:14:57	CREATE TABLE infectious_cases_normalized (id INT AUTO_INCREMENT PRIMARY KEY, entity_id INT NOT NULL, Year INT NO...	0 row(s) affected

рис. 9 Створюємо нормалізовану таблицю `infectious_cases_normalized` `entity_id` | `Year` | всі показники, після чого ми вже не використовуємо більше `Entity` і `Code` напряму, а використовуємо `entity_id` (зовнішній ключ), це прибирає дублювання, пришвидшує запити, відповідає нормам проєктування БД

Крок 2.2 Перенесення даних (JOIN)

Зараз у нас в старій таблиці → `Entity`, `Code`, в новій → потрібен `entity_id` - це значить що треба:

- знайти `entity_id` через JOIN
- вставити всі дані в нову таблицю

Для цього використовуємо команду яка зазначена на рис.10 нижче, після чого всі 10521 рядків перейдуть у нову таблицю, замість тексту буде `entity_id`, база стане нормалізованою.

The screenshot shows the MySQL Workbench interface. On the left, the 'SCHEMAS' pane shows the database structure, including the 'infectious_cases' and 'infectious_cases_normalized' tables. The 'infectious_cases_normalized' table has columns: `id`, `entity_id`, `Year`, `Number_yaws`, `polio_cases`, `cases_guinea_worm`, `Number_rabies`, `Number_malaria`, `Number_hiv`, `Number_tuberculosis`, `Number_smallpox`, and `Number_cholera_cases`.

The SQL editor shows the following query:

```

1  INSERT INTO infectious_cases_normalized (
2      entity_id,
3      Year,
4      Number_yaws,
5      polio_cases,
6      cases_guinea_worm,
7      Number_rabies,
8      Number_malaria,
9      Number_hiv,
10     Number_tuberculosis,
11     Number_smallpox,
12     Number_cholera_cases
13 )
14 SELECT
15     e.id,
16     e.Year,
17     ic.Number_yaws,
18     ic.polio_cases,
19     ic.cases_guinea_worm,
20     ic.Number_rabies,
21     ic.Number_malaria,
22     ic.Number_hiv,
23     ic.Number_tuberculosis,
24     ic.Number_smallpox,
25     ic.Number_cholera_cases
26 FROM infectious_cases ic
27 JOIN entities e
28     ON ic.Entity = e.Entity
29     AND ic.Code = e.Code;

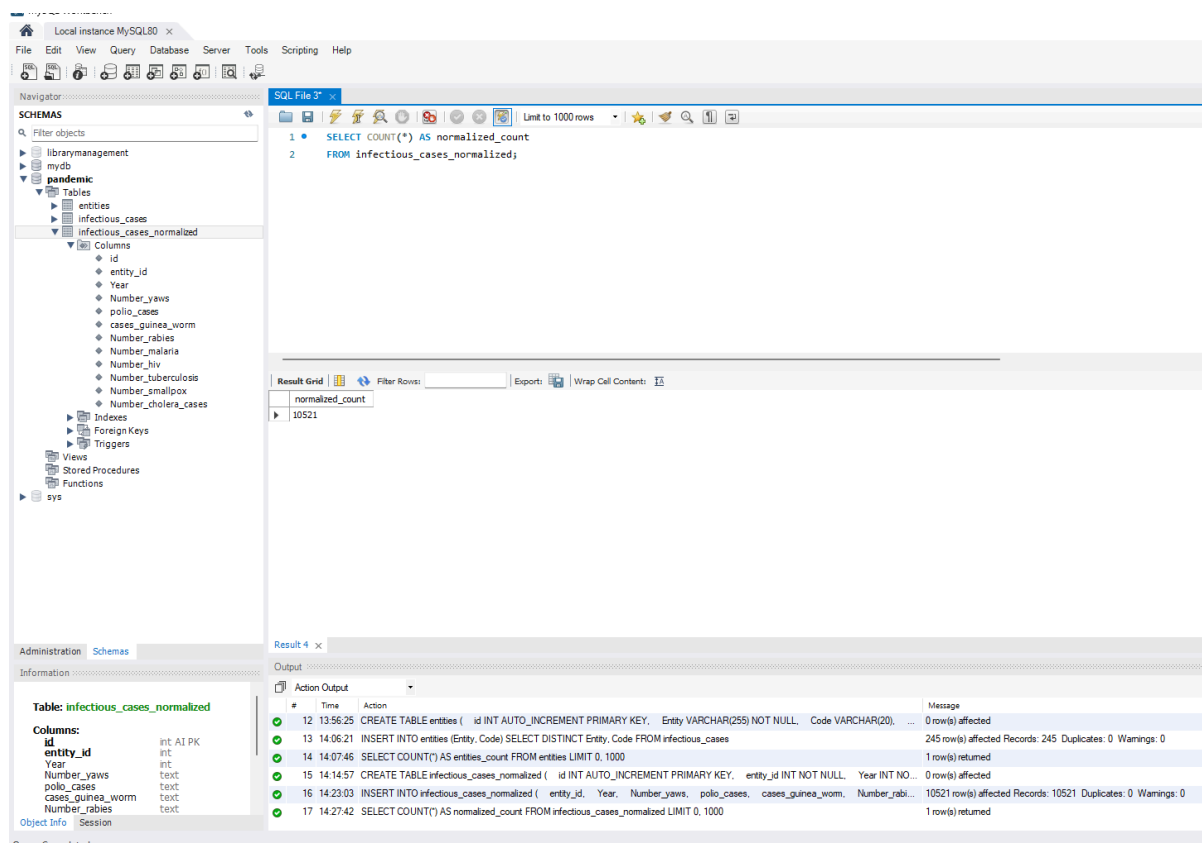
```

The 'Output' pane at the bottom shows the execution results:

#	Time	Action	Message
11	13:56:25	SELECT COUNT(*) AS total_rows FROM infectious_cases LIMIT 0, 1000	1 row(s) returned
12	13:56:25	CREATE TABLE entities (id INT AUTO_INCREMENT PRIMARY KEY, Entity VARCHAR(255) NOT NULL, Code VARCHAR(20), ...)	0 row(s) affected
13	14:06:21	INSERT INTO entities (Entity, Code) SELECT DISTINCT Entity, Code FROM infectious_cases	245 row(s) affected Records: 245 Duplicates: 0 Warnings: 0
14	14:07:46	SELECT COUNT(*) AS entities_count FROM entities LIMIT 0, 1000	1 row(s) returned
15	14:14:57	CREATE TABLE infectious_cases_normalized (id INT AUTO_INCREMENT PRIMARY KEY, entity_id INT NOT NULL, Year INT NO...)	0 row(s) affected
16	14:23:03	INSERT INTO infectious_cases_normalized (entity_id, Year, Number_yaws, polio_cases, cases_guinea_worm, Number_rabi...)	10521 row(s) affected Records: 10521 Duplicates: 0 Warnings: 0

рис. 10 Перенесення даних.

Виконаємо запит `SELECT COUNT(*) FROM infectious_cases` задля перевірки, щоб ментор міг зрозуміти, скільки записів ви завантажили у базу даних із файла.



ріс. 11. 10521 рядок у нормалізованій таблиці → нормалізація завершена правильно (3НФ).

- жоден рядок не загубився
- зв'язок через `entity_id` працює
- структура відповідає **3NF (Third Normal Form — третя нормальна форма)**

Крок 3. Аналітика.

Для кожної унікальної комбінації Entity та Code або їх id порахуйте середнє, мінімальне, максимальне значення та суму для атрибута Number_rabies.

💡 Врахуйте, що атрибут Number_rabies може містити порожні значення " — вам попередньо необхідно їх відфільтрувати. Результат відсортуйте за порашованим середнім значенням у порядку спадання. Оберіть тільки 10 рядків для виведення на екран.

Для кожної країни (Entity + Code) порахуємо:

- AVG (середнє)
- MIN (мінімум)
- MAX (максимум)
- SUM (сума)

Слід зазначити, Number_rabies може бути " , тому: використовуємо **NULLIF**(перетворює " → NULL); використовуємо **CAST** (перетворює TEXT → число); **JOIN entities**

повертаємо назву країни (бо тепер у нас тільки `entity_id`), результат чого відображено на наступному скріні рис. 12

The screenshot shows the MySQL Workbench interface. On the left, the 'SCHEMAS' pane displays the database structure, including tables like 'infectious_cases_normalized' and 'entities'. The main editor shows a SQL query that joins these tables and calculates aggregate statistics for 'rabies' cases. The 'Result Grid' at the bottom displays the query results, showing columns for Entity, Code, and various rabies statistics. The 'Action Output' pane at the bottom shows the execution progress of the query.

Entity	Code	avg_rabies	min_rabies	max_rabies	sum_rabies
World	OVID_WRL	20192.370667	14075.51	24744.66	605771.12
Lower Middle Income (WB)		15193.959333	10202.53	19182.80	458818.78
South Asia (WB)		11729.889667	7271.28	15361.88	351896.69
South-East Asia Region (WHO)		11424.327667	6806.01	15641.96	342729.83
G20		10189.046000	6339.08	13164.88	305671.38
India	IND	8599.172667	5425.87	11121.14	257975.18
Sub-Saharan Africa (WB)		4748.268333	4356.82	5017.98	142448.05
African Region (WHO)		4649.702000	4283.07	4917.78	139491.06
Low Income (WB)		3568.106000	3014.62	3873.53	107043.18
East Asia & Pacific (WB)		3427.461000	2009.94	4991.01	102823.83

рис. 12 JOIN зроблений правильно, NULLIF + CAST використані коректно, GROUP BY + ORDER BY працюють, LIMIT 10 — виконано.

Слід зазначити, що в топі ми маємо World, Low Middle Income (WB), South Asia (WB)... Це не тільки країни, а й агреговані регіони (WHO, World Bank), здається для цього датасету це норм, но при реальній аналітиці все ж краще буде такі речі треба фільтрувати окремо (якщо аналіз тільки по країнах).

Крок 4. Побудуйте колонку різниці в роках.

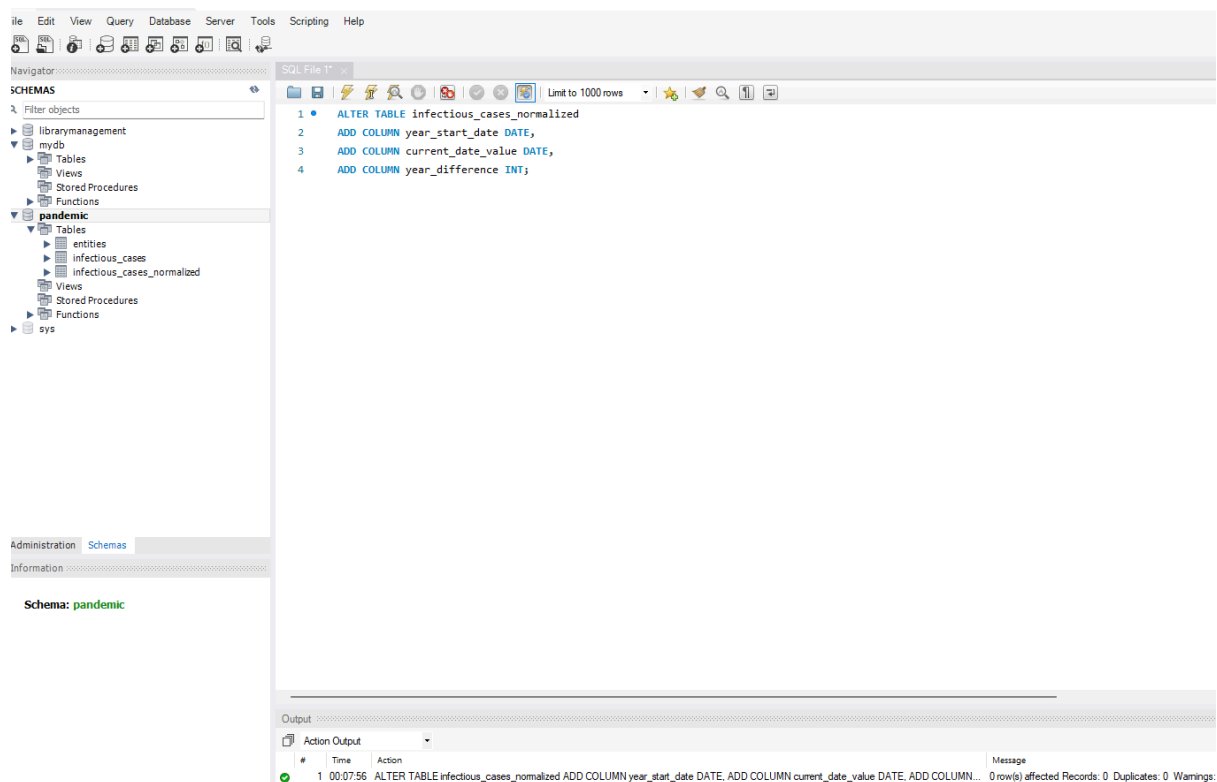
Для оригінальної або нормованої таблиці для колонки Year побудуйте з використанням вбудованих SQL-функцій: атрибут, що створює дату першого січня відповідного року, Наприклад, якщо атрибут містить значення '1996', то значення нового атрибута має бути '1996-01-01'.

атрибут, що дорівнює поточній даті,

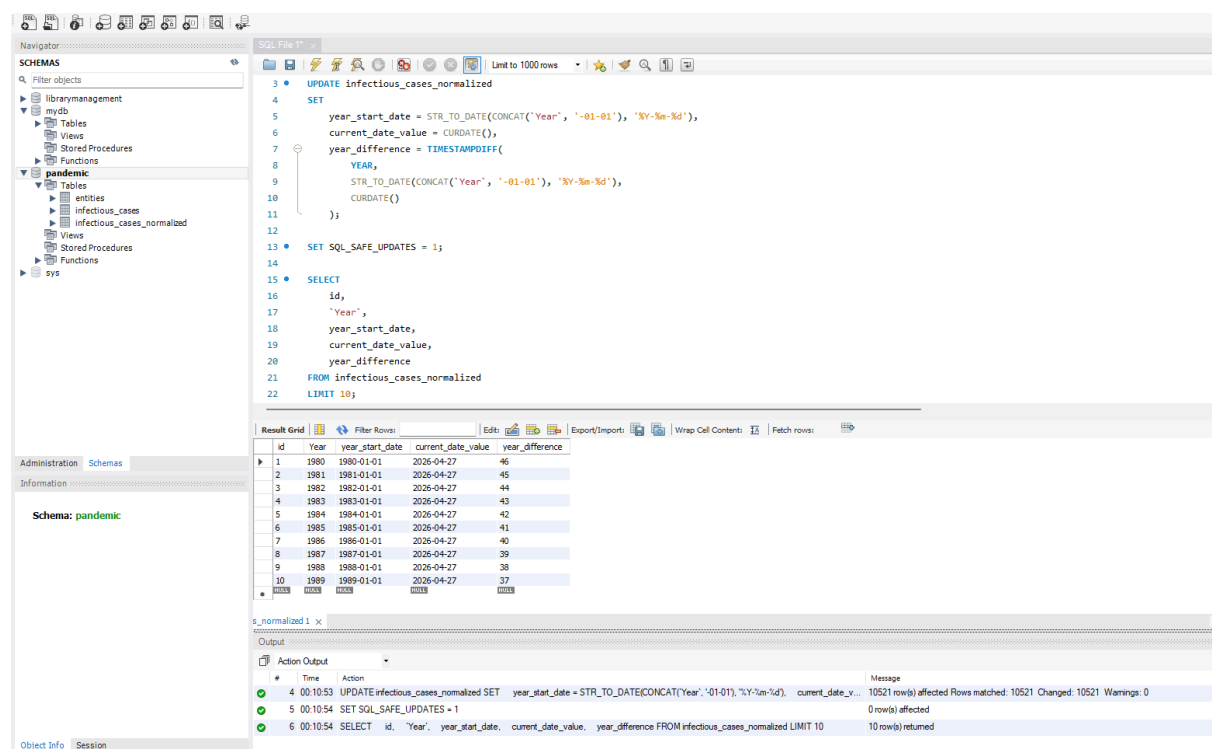
атрибут, що дорівнює різниці в роках двох вищезгаданих колонок.

Перераховувати всі інші атрибути, такі як Number_malaria, не потрібно.

Для пошуку необхідних вбудованих функцій вам може знадобитися матеріал до теми 7.



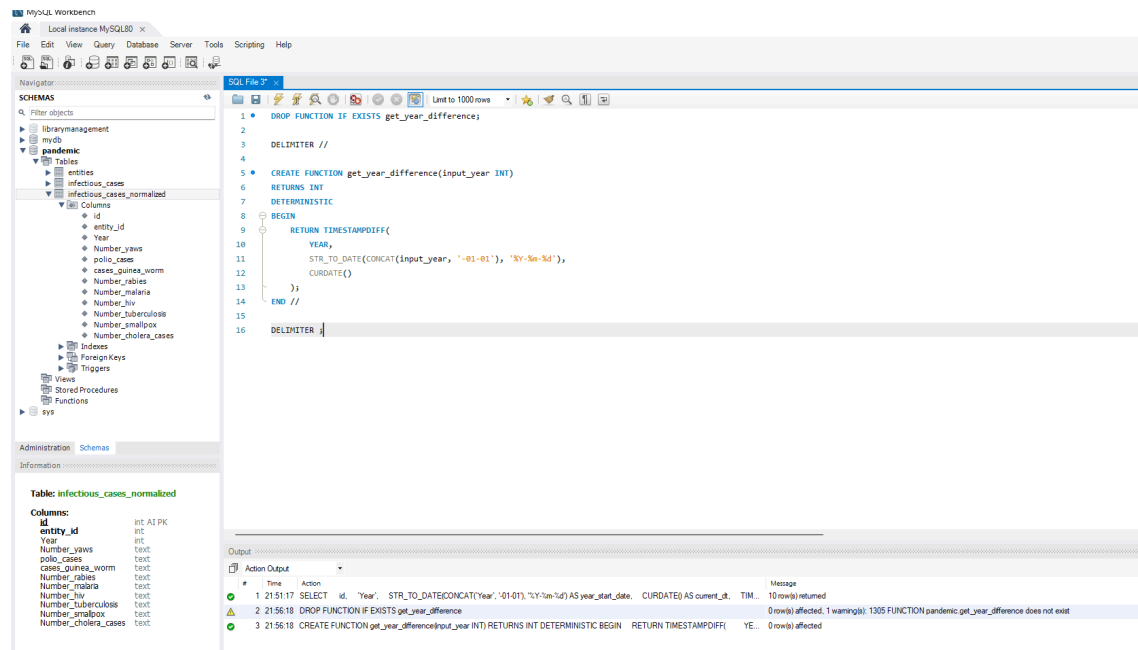
ріс. 13 Створюємо колонки.



ріс. 13.1 Заповнюємо дані

Колонки додані через ALTER TABLE, дані заповнені через UPDATE із використанням функцій STR_TO_DATE та TIMESTAMPDIF.

5. Побудуйте власну функцію.



The screenshot shows the MySQL Workbench interface. On the left, the 'Schemas' pane displays the 'pandemic' database structure, including the 'infectious_cases_normalized' table. The main editor shows the following SQL script:

```

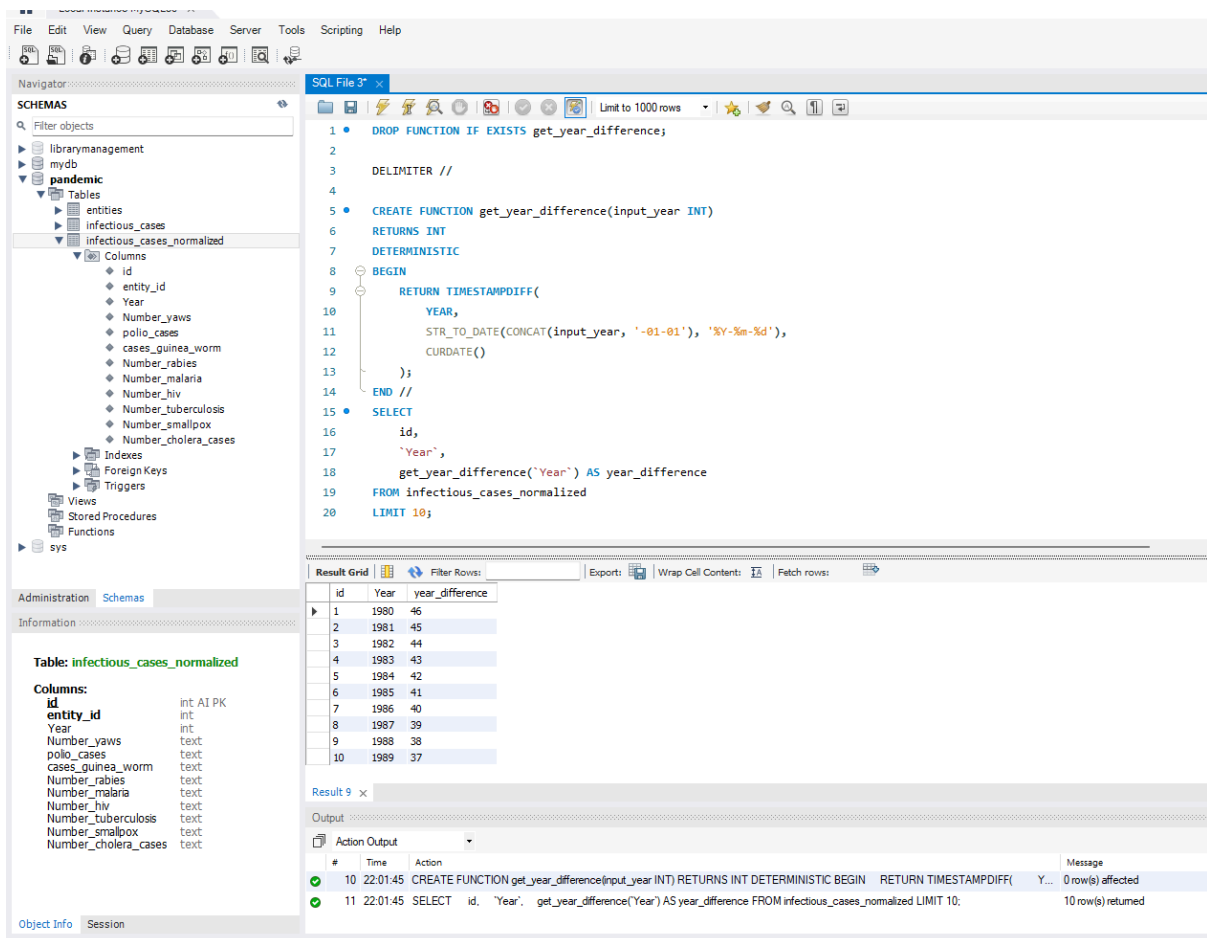
1 DROP FUNCTION IF EXISTS get_year_difference;
2
3 DELIMITER //
4
5 CREATE FUNCTION get_year_difference(input_year INT)
6 RETURNS INT
7 DETERMINISTIC
8 BEGIN
9     RETURN TIMESTAMPDIFF(
10        YEAR,
11        STR_TO_DATE(CONCAT(input_year, '-01-01'), '%Y-%m-%d'),
12        CURDATE()
13    );
14 END //
15
16 DELIMITER ;

```

The 'Output' pane at the bottom shows the execution results:

#	Time	Action	Message
1	21:51:17	SELECT id, 'Year', STR_TO_DATE(CONCAT('Year', '-01-01'), '%Y-%m-%d') AS year_start_date, CURDATE() AS current_date, TIM...	10 row(s) returned
2	21:56:18	DROP FUNCTION IF EXISTS get_year_difference	0 row(s) affected, 1 warning(s): 1305 FUNCTION get_year_difference does not exist
3	21:56:18	CREATE FUNCTION get_year_difference(input_year INT) RETURNS INT DETERMINISTIC BEGIN RETURN TIMESTAMPDIFF(	0 row(s) affected

pic.14



The screenshot shows the MySQL Workbench interface with the same SQL script as in the previous image. The 'Result Grid' pane displays the results of the query:

id	Year	year_difference
1	1980	46
2	1981	45
3	1982	44
4	1983	43
5	1984	42
6	1985	41
7	1986	40
8	1987	39
9	1988	38
10	1989	37

The 'Output' pane at the bottom shows the execution results:

#	Time	Action	Message
10	22:01:45	CREATE FUNCTION get_year_difference(input_year INT) RETURNS INT DETERMINISTIC BEGIN RETURN TIMESTAMPDIFF(	0 row(s) affected
11	22:01:45	SELECT id, 'Year', get_year_difference('Year') AS year_difference FROM infectious_cases_normalized LIMIT 10;	10 row(s) returned

pic.15

Створіть і використайте функцію, що будує такий же атрибут, як і в попередньому завданні: функція має приймати на вхід значення року, а повертати різницю в роках між поточною датою та датою, створеною з атрибута року (1996 рік → '1996-01-01').

💡 Якщо ви не виконали попереднє завдання, то можете побудувати іншу функцію — функцію, що рахує кількість захворювань за певний період. Для цього треба поділити кількість захворювань на рік на певне число: 12 — для отримання середньої кількості захворювань на місяць, 4 — на квартал або 2 — на півріччя. Таким чином, функція буде приймати два параметри: кількість захворювань на рік та довільний дільник. Ви також маєте використати її — запустити на даних. Оскільки не всі рядки містять число захворювань, вам необхідно буде відсіяти ті, що не мають чисельного значення ($\neq$ “”).

The screenshot shows the MySQL Workbench interface. On the left, the 'SCHEMAS' pane displays the database structure, including the 'infectious_cases_normalized' table. The main editor shows the SQL script for creating the function. The 'Output' pane at the bottom shows the execution results, including a warning about a non-existent function and the successful creation of the new function.

```

1 DROP FUNCTION IF EXISTS calc_cases_per_period;
2
3 DELIMITER //
4
5 CREATE FUNCTION calc_cases_per_period(
6     yearly_cases DECIMAL(20,2),
7     divisor INT
8 )
9 RETURNS DECIMAL(20,2)
10 DETERMINISTIC
11 BEGIN
12     -- Захист від ділення на 0 або NULL
13     IF yearly_cases IS NULL OR divisor IS NULL OR divisor = 0 THEN
14         RETURN NULL;
15     END IF;
16
17     RETURN yearly_cases / divisor;
18 END //
19
20 DELIMITER ;

```

Table: infectious_cases_normalized

Columns:		
id	int	AI PK
entity_id	int	
Year	int	
Number_yaws	text	
polio_cases	text	
cases_guinea_worm	text	
Number_rabies	text	
Number_malaria	text	
Number_hiv	text	
Number_tuberculosis	text	
Number_smallpox	text	
Number_cholera_cases	text	

Output

Action	Time	Action	Message
11	22:01:45	SELECT id, 'Year', get_year_difference('Year') AS year_difference FROM infectious_cases_normalized LIMIT 10;	10 row(s) returned
12	22:08:51	DROP FUNCTION IF EXISTS calc_cases_per_period;	0 row(s) affected, 1 warning(s): 1305 FUNCTION pandemic.calc_cases_per_period does not exist
13	22:08:51	CREATE FUNCTION calc_cases_per_period(yearly_cases DECIMAL(20,2), divisor INT) RETURNS DECIMAL(20,2) DETERMINISTIC;	0 row(s) affected
14	22:08:54	DROP FUNCTION IF EXISTS calc_cases_per_period;	0 row(s) affected
15	22:08:54	CREATE FUNCTION calc_cases_per_period(yearly_cases DECIMAL(20,2), divisor INT) RETURNS DECIMAL(20,2) DETERMINISTIC;	0 row(s) affected

pic.16 Рахує середню кількість випадків за період (місяць/квартал/півріччя) → $\text{cases_per_period} = \text{yearly_cases} / \text{divisor}$.

- **yearly_cases** — вже число (ми його підготуємо в запиті через **CAST**)
- **divisor** — 12/4/2
- є базова валідація (NULL / 0)

Використовуємо функцію на даних:

Table: infectious_cases_normalized

Columns:

Column	DataType	Attributes
id	int	AI PK
entity_id	int	
Year	int	
Number_yaws	text	
polio_cases	text	
cases_guinea_worm	text	
Number_rabies	text	
Number_malaria	text	
Number_hiv	text	
Number_tuberculosis	text	
Number_smallpox	text	
Number_cholera_cases	text	

SQL Script:

```

12 -- Захист від ділення на 0 або NULL
13 IF yearly_cases IS NULL OR divisor IS NULL OR divisor = 0 THEN
14     RETURN NULL;
15 END IF;
16
17 RETURN yearly_cases / divisor;
18
19
20 DELIMITER ;
21 • SELECT
22     id,
23     "Year",
24     calc_cases_per_period(
25         CAST(NULLIF(Number_rabies, '') AS DECIMAL(20,2)),
26         12
27     ) AS avg_per_month
28 FROM infectious_cases_normalized
29 WHERE NULLIF(Number_rabies, '') IS NOT NULL
30 LIMIT 10;
  
```

Result Grid:

#	id	Year	avg_per_month
11	1990	0.19	
12	1991	0.19	
13	1995	0.26	
14	1997	0.29	
15	1998	0.30	
16	1999	0.30	
17	2000	0.30	
18	2001	0.31	
19	2002	0.32	
20	2003	0.33	

Action Output:

#	Time	Action	Message
14	22:08:54	DROP FUNCTION IF EXISTS calc_cases_per_period	0 row(s) affected
15	22:08:54	CREATE FUNCTION calc_cases_per_period(yearly_cases DECIMAL(20,2), divisor INT) RETURNS DECIMAL(20,2) DETERMINISTIC	0 row(s) affected
16	22:11:40	DROP FUNCTION IF EXISTS calc_cases_per_period	0 row(s) affected
17	22:11:40	CREATE FUNCTION calc_cases_per_period(yearly_cases DECIMAL(20,2), divisor INT) RETURNS DECIMAL(20,2) DETERMINISTIC	0 row(s) affected
18	22:11:40	SELECT id, 'Year', calc_cases_per_period(CAST(NULLIF(Number_rabies, '') AS DECIMAL(20,2)), 12) AS avg_per_month	10 row(s) returned

pic.17 ВАРИАНТ А — середнє на місяць (rabies)

Table: infectious_cases_normalized

Columns:

Column	DataType	Attributes
id	int	AI PK
entity_id	int	
Year	int	
Number_yaws	text	
polio_cases	text	
cases_guinea_worm	text	
Number_rabies	text	
Number_malaria	text	
Number_hiv	text	
Number_tuberculosis	text	
Number_smallpox	text	
Number_cholera_cases	text	

SQL Script:

```

12 -- Захист від ділення на 0 або NULL
13 IF yearly_cases IS NULL OR divisor IS NULL OR divisor = 0 THEN
14     RETURN NULL;
15 END IF;
16
17 RETURN yearly_cases / divisor;
18
19
20 DELIMITER ;
21 • SELECT
22     id,
23     "Year",
24     calc_cases_per_period(
25         CAST(NULLIF(Number_rabies, '') AS DECIMAL(20,2)),
26         4
27     ) AS avg_per_quarter
28 FROM infectious_cases_normalized
29 WHERE NULLIF(Number_rabies, '') IS NOT NULL
30 LIMIT 10;
  
```

Result Grid:

#	id	Year	avg_per_quarter
11	1990	0.54	
12	1991	0.58	
13	1995	0.78	
14	1997	0.86	
15	1998	0.89	
16	1999	0.90	
17	2000	0.91	
18	2001	0.93	
19	2002	0.96	
20	2003	0.98	

Action Output:

#	Time	Action	Message
25	22:13:51	CREATE FUNCTION calc_cases_per_period(yearly_cases DECIMAL(20,2), divisor INT) RETURNS DECIMAL(20,2) DETERMINISTIC	0 row(s) affected
27	22:13:51	SELECT id, 'Year', calc_cases_per_period(CAST(NULLIF(Number_rabies, '') AS DECIMAL(20,2)), 2) AS avg_per_half_year	10 row(s) returned
28	22:14:44	DROP FUNCTION IF EXISTS calc_cases_per_period	0 row(s) affected
29	22:14:44	CREATE FUNCTION calc_cases_per_period(yearly_cases DECIMAL(20,2), divisor INT) RETURNS DECIMAL(20,2) DETERMINISTIC	0 row(s) affected
30	22:14:44	SELECT id, 'Year', calc_cases_per_period(CAST(NULLIF(Number_rabies, '') AS DECIMAL(20,2)), 4) AS avg_per_quarter	10 row(s) returned

pic.18 ВАРИАНТ В — середнє на квартал

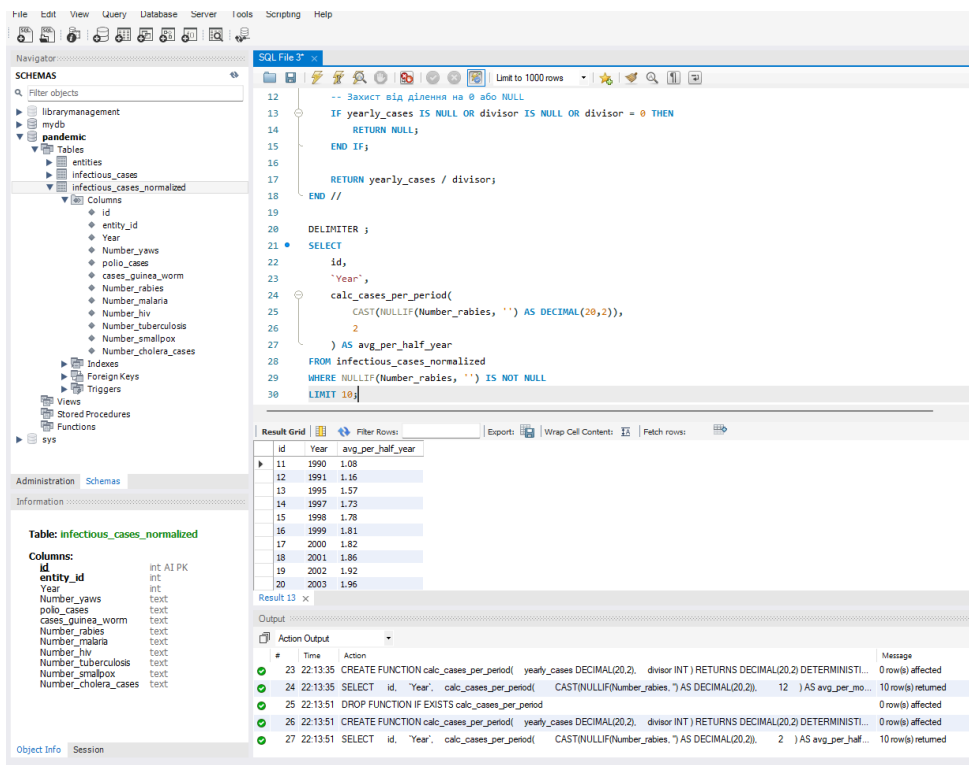


рис.19 ВАРІАНТ С — середнє на півріччя

Додатково було реалізовано власну функцію `calc_cases_per_period`, яка дозволяє обчислювати середню кількість захворювань за різні періоди (місяць, квартал, півріччя) шляхом параметризації дільника. Перед обчисленням було виконано очищення даних від порожніх значень та приведення типів, що забезпечує коректність результатів і можливість повторного використання функції в аналітичних запитах.

Фінальний висновок.

У ході виконання фінального проекту було створено схему бази даних, імпортовано дані та проведено їх нормалізацію до третьої нормальної форми (3NF). Повторювані атрибути було винесено в окрему таблицю, що дозволило зменшити надлишковість і підвищити цілісність даних.

Було реалізовано аналітичні SQL-запити для обчислення статистичних показників, а також виконано обробку дат із використанням вбудованих функцій MySQL.

Додатково було створено власну функцію для обчислення різниці в роках, що дозволяє повторно використовувати бізнес-логіку в запитах.